Universidad Rafael Landívar

Quetzaltenango

Facultad de Ingeniería

Departamento de Informática y Sistemas

Estructura de Datos II

Ing. Manuel Gustavo Rojas Vásquez

MANUAL TÉCNICO

Morelia Gonzalez Ixcot 1536325

Keila Belinda Puac Gómmez Carne: 1660625

Jackelin SofiaVásquez Guzmán Carné 1503725

05 de Noviembre de 2026

Índice

Introducción 1

Manual Técnico: Proyecto Red de Transporte 1

Alcance y Descripción del Sistema 1

Diseño General y Arquitectura 1

Diagramas del Programa 2

Diagrama de Flujo del Progama 2

Diagrama Inicio y Menú Principal 3

Diagrama de Flujo Registro de una parada 4

Diagrama de Flujo Búsqueda de una Parada 4

Diagrama de Flujo Calculo ruta mas corta de grafo y Dijstra 5

Diagrama de Flujo de Cálculo de Ruta 5

Diagrama de Flujo registro de paradas y rutas 6

Diagrama de Flujo Buscar sugerencias de paradas (Trie) 6

Diagrama de Clases 7

Diagrama Clases internas de Estructuras 8

Explicación de Implementación y Justificación de Estructura de Datos 9

Explicación de la Implementación del Sistema 9

Implementación de la Tabla Hash 9

Implementación del Grafo 10

Implementación del Union-Find 10

Implementación del Trie 11

Implementación de Estructuras 12

Carga de Datos y Archivos CSV 12

Justificación de Estructura de Datos 12

Análisis de Complejidad de Estructuras..... 15

Estructuras y sus Ventajas 19

Introducción

Este manual describe el diseño y la implementación de la aplicación Red de Transporte Urbano, desarrollada en C# para el Proyecto del curso Estructura de Datos II. El sistema administra las paradas y rutas de buses de la ciudad (los datos de ejemplo corresponden a paradas de Quetzaltenango, Almolonga y Salcajá), permite localizar una parada por su código, autocompletar nombres mientras se escribe, calcular la ruta más rápida o más económica entre dos paradas y verificar si dos paradas pertenecen a la misma zona conectada de la red.

El manual presenta la función de cada estructura, su integración, los algoritmos de recorrido y búsqueda de rutas, los diagramas y un análisis de complejidad en notación Big O.

Manual Técnico: Proyecto Red de Transporte

Alcance y Descripción del Sistema

El sistema se enfoca en una red de transporte representada por paradas y rutas. Cada parada cuenta con un código, nombre, latitud y longitud propia. Cada ruta relaciona una parada de origen con una parada de destino y almacena el tiempo estimado, costo y condición de sentido único o bidireccional.

Dentro de las funciones que realiza el programa se encuentra:

- Registrar y eliminar paradas.
- Registrar y eliminar rutas.
- Cargar paradas y rutas desde archivos CSV.
- Buscar una parada por código o por nombre.
- Mostrar sugerencias de autocompletar a partir de prefijo.
- Consultas zonas conectadas.
- Ejecutar recorrido BFS y DFS.
- Calcular la ruta de menor tiempo o costo.
- Visualizar información de la red y de la estructura.

Diseño General y Arquitectura

El programa se encuentra organizado de manera que cada archivo cumple con una responsabilidad única, esto para evitar errores y a su vez garantizar que en un futuro pueda ser escalable.

Diagramas del Programa

Diagrama de Flujo del Progama

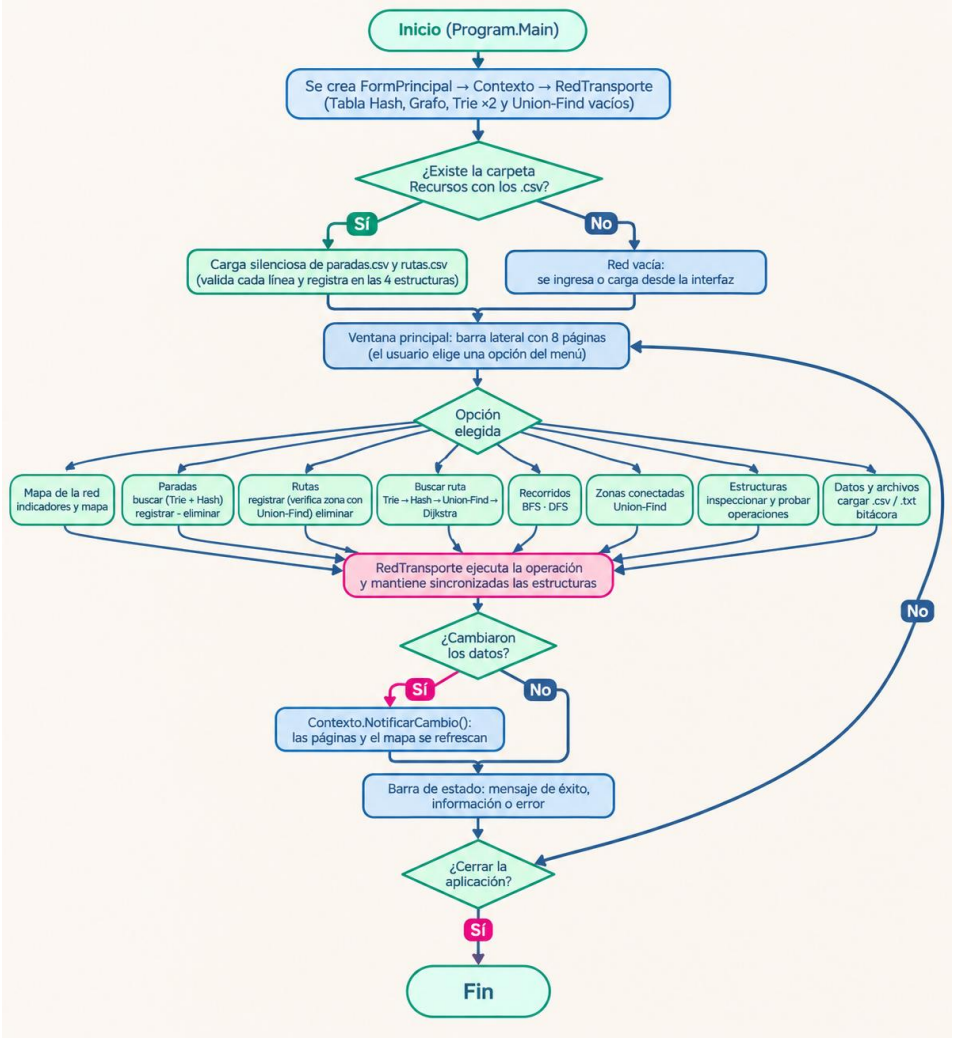


Diagrama Inicio y Menú Principal

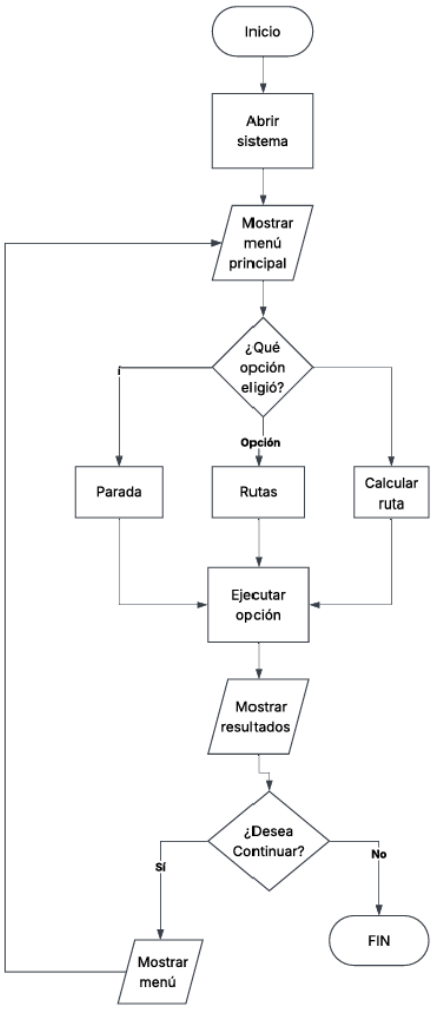


Diagrama de Flujo Registro de una parada

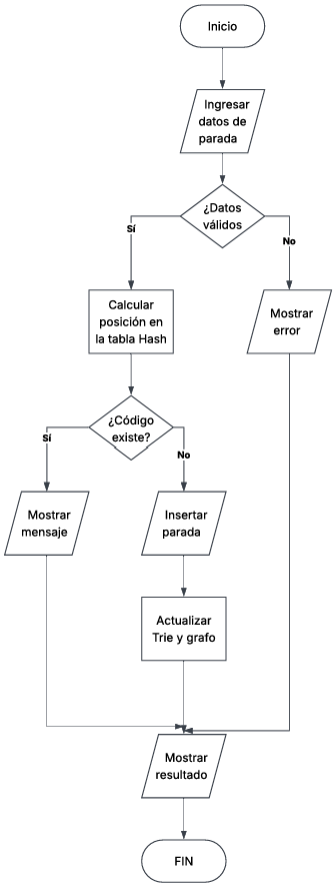


Diagrama de Flujo Búsqueda de una Parada

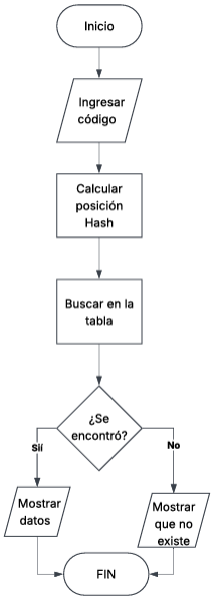


Diagrama de Flujo Calculo ruta mas corta de grafo y Dijstra

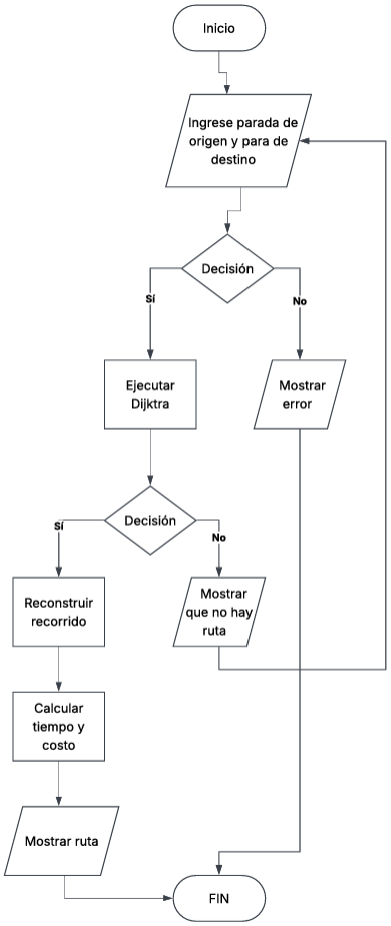


Diagrama de Flujo de Cálculo de Ruta

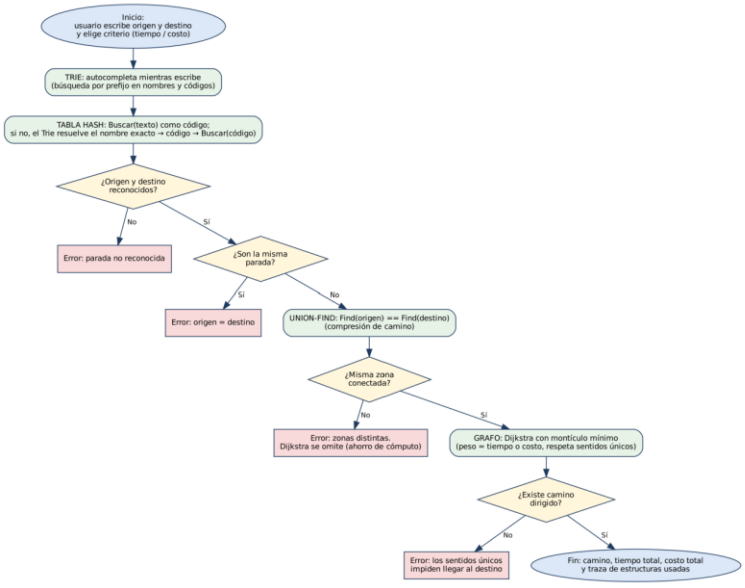


Diagrama de Flujo registro de paradas y rutas

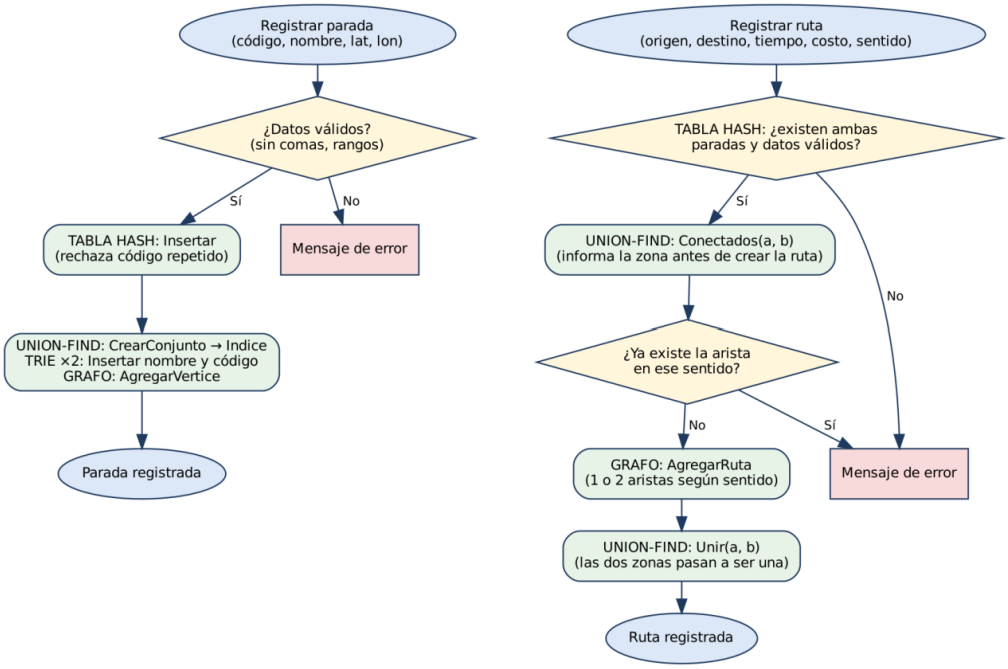


Diagrama de Flujo Buscar sugerencias de paradas (Trie)

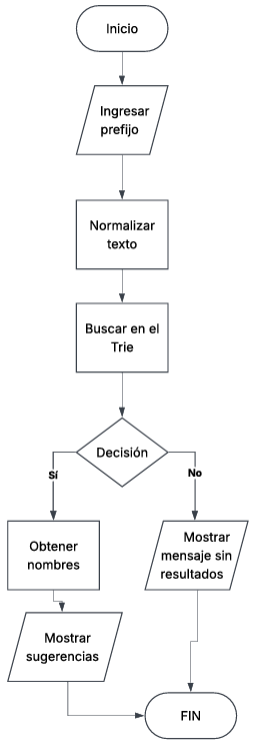
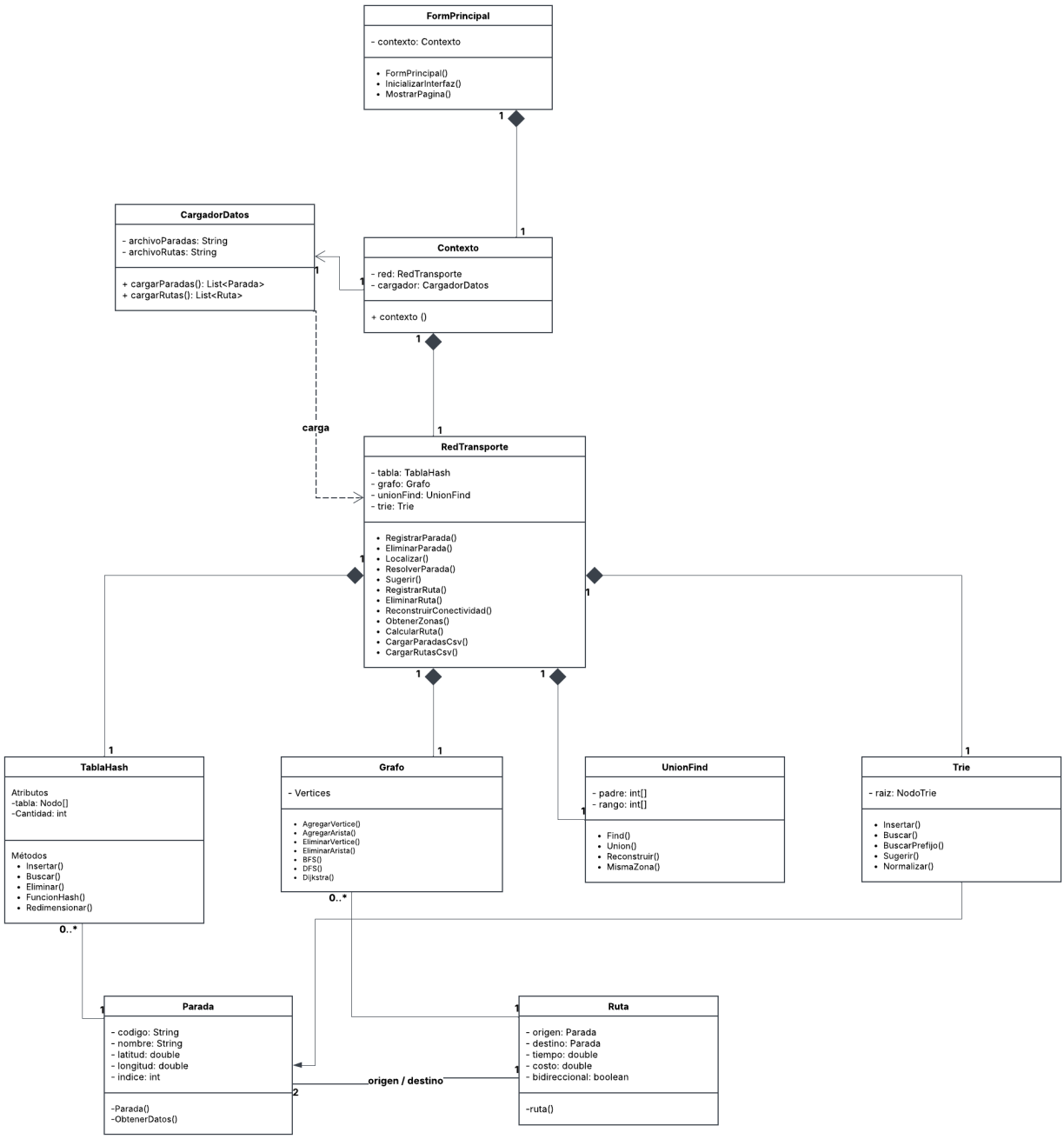


Diagrama de Clases

Diagrama de Clases – Sistema de Transporte Urbano.



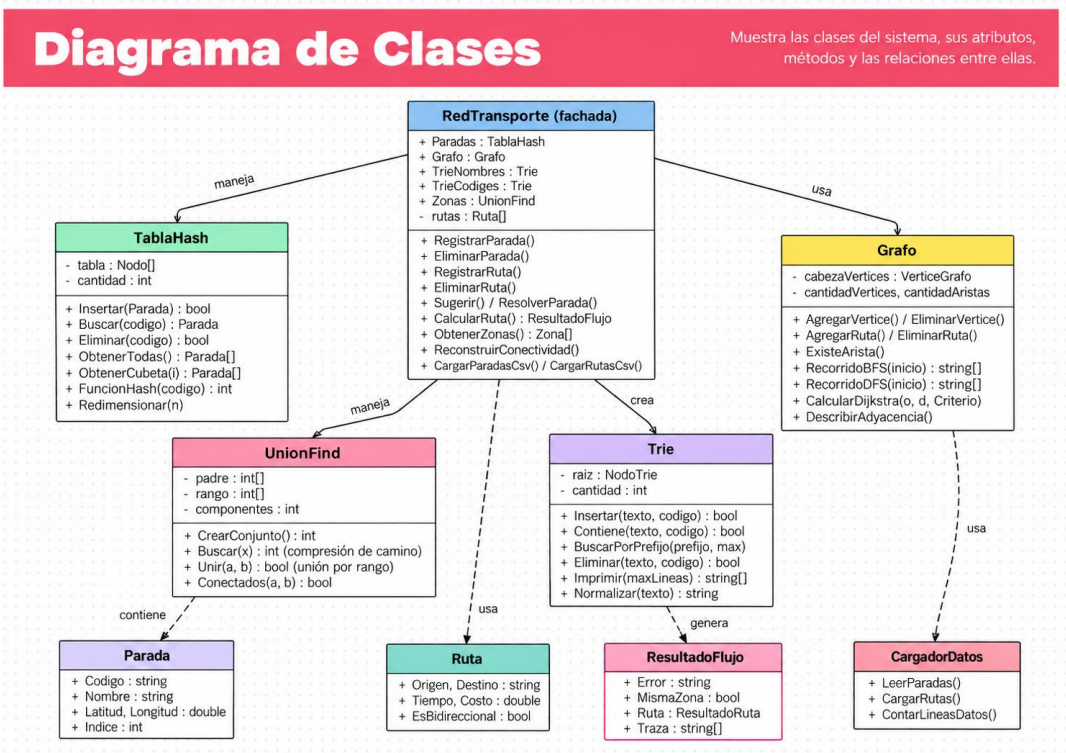
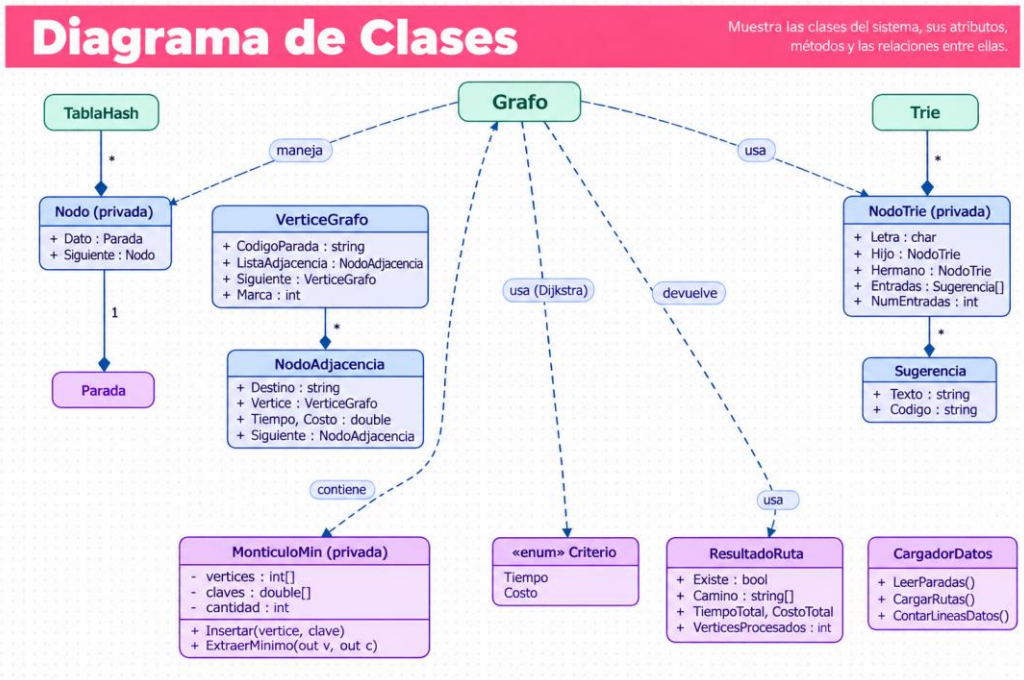


Diagrama Clases internas de Estructuras



Explicación de Implementación y Justificación de Estructura de Datos

Explicación de la Implementación del Sistema

El proyecto fue desarrollado completamente en C#, con una interfaz gráfica y un diseño modular dividido en capas: modelos (Parada y Ruta), estructuras de datos, servicios (RedTransporte y carga de archivos) e interfaz. La implementación de las cuatro estructuras se realizó de forma manual, utilizando arreglos y referencias entre nodos, además de métodos como inserción, búsqueda, eliminación y recorrido.

Implementación de la Tabla Hash

La Tabla Hash se encarga de guardar el registro completo de cada parada y permite localizarla por su código único mediante encadenamiento para resolver colisiones.

- **Estructura de los nodos:** la tabla es un arreglo de cubetas (Nodo[]). Cada cubeta es una lista enlazada simple cuyos nodos guardan una Parada y la referencia al siguiente nodo. La capacidad inicial es 17, un número primo, para distribuir mejor los códigos.
- **Función hash:** es polinómica con base 31 sobre el código normalizado (quiere decir que esta sin espacios y en mayúsculas). El resultado se reduce módulo la capacidad en cada paso para evitar desbordamientos.
- **Inserción y redimensionamiento:** antes de insertar, se recorre la cadena para rechazar códigos repetidos. Si el factor de carga supera el 0.75, la tabla duplica su capacidad (2n + 1) y reubica todos los nodos (realiza un rehash). El nuevo nodo se coloca al inicio de la cadena. Con los 13 registros de ejemplo la tabla crece de 17 a 35 al insertar la parada P13.
- **Búsqueda y eliminación:** se calcula la cubeta con la función hash y se recorre su cadena comparando códigos. Para eliminar, el nodo se desenlaza de la cadena ajustando la referencia del nodo anterior.
- **Recorrido e impresión**: La función ObtenerTodas recorre y devuelve las paradas; ObtenerCubeta e IndiceCubeta permiten a la interfaz mostrar en qué posición quedó cada código; Mostrar imprime la tabla completa con sus cadenas.

Implementación del Grafo

El grafo es dirigido y ponderado, además se encarga de modelar la red de buses, donde cada parada es un vértice y cada ruta es una arista con dos pesos, tiempo y costo.

- **Estructura de los nodos:** se utiliza una lista de adyacencia con nodos enlazados. Una lista enlazada de VerticeGrafo (con referencia a la cabeza y a la cola para insertar al final) y dentro de cada vértice, una lista enlazada de NodoAdyacencia que guarda el destino, el tiempo, el costo y una referencia directa al vértice destino. Una ruta bidireccional se guarda como dos aristas opuestas y una de sentido único como una sola.
- **Recorrido BFS:** antes de cada recorrido, el método Indexar asigna a cada vértice un número de 0 a V-1 (campo Marca) para poder usar un arreglo de visitados. El BFS visita los vértices por niveles usando una Queue únicamente como control del orden de visita, respetando el sentido de las aristas.
- **Recorrido DFS:** se implementó de forma recursiva, marca el vértice, lo agrega al orden de visita y continúa con el primer vecino no visitado antes de retroceder.
- **Algoritmo de Dijkstra:** calcula el camino de menor tiempo o de menor costo según el criterio elegido. Usa un montículo binario mínimo propio (MonticuloMin) con arreglos de vértices y claves 8 que se duplican cuando se llenan. En lugar de decrementar claves, inserta una entrada nueva cada vez que mejora una distancia y descarta las entradas obsoletas al extraerlas. El algoritmo se detiene cuando el destino se vuelve definitivo y el camino se reconstruye con el arreglo de predecesores, sumando tiempo y costo totales.
- **Eliminación:** EliminarRuta quita la arista (y su inversa si era bidireccional) y la función EliminarVertice quita el vértice junto con todas las aristas que entran o salen de él.

Implementación del Union-Find

La estrucutura de Union-Find es la encargada de mantener las zonas conectadas de la red. Dos paradas pertenecen a la misma zona si existe una cadena de rutas que las une.

- **Almacenamiento:** se utilizan dos arreglos de enteros, padre[], que representa un bosque de árboles donde cada raíz se apunta a sí misma, y rango[], que guarda una

cota de la altura de cada árbol. Ambos duplican su tamaño cuando se llenan, cada parada recibe un índice cuando se crea su conjunto con CrearConjunto.

- **Búsqueda con compresión de camino:** el método Buscar recorre el camino hasta la raíz y luego hace una segunda pasada que deja a todos los nodos visitados apuntando directamente a ella, lo que acorta las búsquedas posteriores.
- **Unión por rango:** el método Unir busca las raíces de ambos elementos, si son distintas, el árbol de menor rango se cuelga del de mayor rango, y si son iguales se elige uno y su rango aumenta en uno. Conectados indica si dos elementos comparten raíz.
- **Reconstrucción:** el Union-Find no puede separar conjuntos. Por eso, cuando se elimina una ruta o una parada, RedTransporte crea uno nuevo y vuelve a unir las paradas con las rutas que siguen vigentes.

Implementación del Trie

El Trie permite el autocompletado de nombres y códigos de parada mientras el usuario escribe. El sistema usa dos instancias: TrieNombres y TrieCodigos.

- **Estructura de los nodos:** cada NodoTrie guarda un carácter, una referencia al primer hijo, una referencia al siguiente hermano y un arreglo dinámico de Sugerencia (texto original y código de la parada) para las palabras que terminan en ese nodo, lo que permite que dos paradas tengan el mismo nombre. Los hijos forman una lista enlazada ordenada de forma alfabética, por lo que no se necesita ningún diccionario y las sugerencias salen ya ordenadas.
- **Normalización:** antes de insertar o buscar, el texto se convierte a minúsculas y se le quitan las tildes, de modo que “café” encuentre “Café”.
- **Inserción:** se desciende carácter por carácter creando, en su posición alfabética, los hijos que no existan. En el último nodo se agrega la sugerencia, rechazando códigos repetidos.
- **Búsqueda por prefijo:** se desciende hasta el nodo que corresponde al prefijo y desde allí se recolectan las sugerencias en profundidad, en orden alfabético, hasta alcanzar el máximo solicitado.
- **Eliminación:** se borra la entrada del último nodo y, de regreso en la recursión, se podan los nodos que quedaron sin palabras ni hijos.

Implementación de Estructuras

RedTransporte se encarga de coordinar las cuatro estructuras para que ninguna quede aislada. Al registrar una parada, la Tabla Hash la inserta (y rechaza duplicados a la vez), el Union-Find crea un conjunto nuevo y guarda el índice en la parada, los dos Tries indexan su nombre y su código, y el Grafo agrega un vértice. Al registrar una ruta, la Tabla Hash confirma que ambas paradas existen, el Grafo verifica que la arista no exista y la agrega, y el Union-Find une las dos zonas, la interfaz consulta antes el Union-Find para informar si las paradas ya estaban conectadas. Para autocompletar, ambos Tries devuelven códigos por prefijo y la Tabla Hash convierte cada código en la parada completa. Para calcular una ruta, el Trie resuelve el texto escrito, la Tabla Hash localiza el registro, el Union-Find verifica la zona (si son zonas distintas, Dijkstra no se ejecuta) y el Grafo calcula el camino. El resultado incluye una traza de qué estructura hizo cada paso.

Carga de Datos y Archivos CSV

Los datos no están escritos en el código, por ello se cargan desde archivos .csv/.txt o se ingresan manualmente en la interfaz.

Lectura del archivo: se utiliza File.ReadAllLines, la primera línea es el encabezado y cada línea restante es una parada (codigo,nombre,latitud,longitud) o una ruta (origen,destino,tiempo,costo,bidireccional).

Procesamiento: cada línea se separa con .Split(',') y los números se convierten con cultura invariante (punto decimal). Las líneas con formato incorrecto se descartan y el motivo se muestra en la bitácora de la interfaz.

Validación: RedTransporte valida cada registro con las mismas reglas del ingreso manual, código y nombre obligatorios, coordenadas en rango, códigos no repetidos, paradas existentes para cada ruta, tiempo mayor que cero y costo no negativo. Esta última condición es la que garantiza la corrección de Dijkstra

Justificación de Estructura de Datos

Tabla Hash:

Se seleccionó la Tabla Hash para almacenar y localizar las paradas del sistema, debido a que una de las operaciones principales de Rutas Xela es consultar una parada mediante su código de identificación. Como cada parada posee un código único, este puede utilizarse

como clave para acceder directamente a la cubeta correspondiente mediante una función hash, evitando recorrer todos los registros para encontrar uno específico.

Esta estructura permite realizar operaciones de búsqueda, inserción y eliminación con una complejidad promedio de O(1), siempre que la distribución de las claves sea adecuada y se controle el factor de carga. En la implementación se utiliza encadenamiento separado para resolver colisiones y redimensionamiento de la tabla cuando el factor de carga alcanza el límite establecido de 0.75.

En comparación con una lista, cuya búsqueda requiere O(n) en el peor caso, la Tabla Hash ofrece un acceso más eficiente conforme aumenta la cantidad de paradas.

Grafo:

Se eligió un Grafo para representar la red de transporte, ya que esta se compone de paradas que se encuentran conectadas mediante rutas. Cada parada representa un vértice y cada conexión entre dos paradas representa una arista, a la cual se le asigna un peso que representa el tiempo o costo del recorrido.

El sistema utiliza un grafo dirigido y ponderado, debido a que no todas las rutas necesariamente permiten desplazarse en ambos sentidos y porque las conexiones tienen valores asociados que permiten calcular recorridos. Para representar las conexiones se utiliza una lista de adyacencia, ya que cada parada se relaciona únicamente con un número limitado de otras paradas. Esta representación resulta más conveniente que una matriz de adyacencia, pues requiere $O(V + E)$ de memoria, mientras que una matriz necesita $O(V^2)$.

A partir del grafo se implementan los algoritmos BFS y DFS, ambos con complejidad $O(V + E)$, para recorrer y explorar la red. También se utiliza Dijkstra con un montículo mínimo propio para determinar rutas de menor tiempo o costo, con una complejidad de $O((V + E) \log V)$. De esta manera, el grafo permite representar la estructura de transporte y realizar los cálculos necesarios para encontrar recorridos entre paradas.

Union-Find:

Se incorporó la estructura Union-Find para administrar y consultar la conectividad entre las paradas de la red. Su función consiste en determinar si dos paradas pertenecen al mismo conjunto o componente conectado, lo que permite identificar si existe una relación de conectividad entre ellas antes de realizar determinadas operaciones de búsqueda de rutas.

Se eligió esta estructura porque permite realizar consultas de conectividad de manera más eficiente que recorrer el grafo mediante BFS o DFS cada vez que se necesita comprobar si dos paradas están conectadas. La implementación utiliza dos optimizaciones: compresión de camino y unión por rango. Gracias a ellas, las operaciones de búsqueda de representante y unión tienen una complejidad amortizada de $O(\alpha(n))$, donde $\alpha(n)$ es la función inversa de Ackermann, cuyo crecimiento es extremadamente lento y que, en la práctica, se comporta como una constante.

Una característica importante de Union-Find es que no permite separar directamente dos elementos que ya pertenecen al mismo conjunto. Por esta razón, cuando se eliminan conexiones de la red, es necesario reconstruir la estructura a partir de las rutas que permanecen activas. Este proceso tiene una complejidad aproximada de $O(P + R \cdot \alpha(P))$, donde P representa la cantidad de paradas y R la cantidad de rutas. A pesar de esta limitación, resulta conveniente porque las consultas de conectividad pueden realizarse de forma muy eficiente.

Trie:

Se seleccionó la estructura Trie para implementar la búsqueda por prefijos y el autocompletado de nombres y códigos de paradas. Esta estructura organiza la información carácter por carácter, permitiendo localizar el nodo correspondiente a un prefijo sin necesidad de comparar individualmente todos los nombres almacenados.

Su principal ventaja es que el tiempo de búsqueda e inserción depende de la longitud del texto consultado, representada por m, y no directamente de la cantidad total de paradas. En la implementación, estas operaciones tienen una complejidad de $O(m \cdot \sigma)$, donde σ representa la cantidad de hijos que puede tener cada nodo. Debido a que los hijos se mantienen ordenados alfabéticamente, las sugerencias pueden generarse siguiendo ese orden.

En comparación con una lista, donde sería necesario revisar los nombres uno por uno mediante operaciones como StartsWith, el Trie permite realizar búsquedas por prefijo de manera más directa. Asimismo, una Tabla Hash no resulta adecuada para esta tarea, ya que está diseñada principalmente para búsquedas por claves completas y no para localizar coincidencias parciales.

Por estas razones, el Trie mejora la experiencia de búsqueda del usuario al permitirle escribir una parte del nombre o código de una parada y obtener sugerencias relacionadas.

Las cuatro estructuras se seleccionaron porque cada una resuelve una necesidad específica dentro del sistema y al trabajar de manera integrada, permiten realizar las operaciones de forma organizada y eficiente.
El Trie facilita la búsqueda de paradas mediante nombres o prefijos; la Tabla Hash permite recuperar rápidamente la información de una parada a partir de su código; Union-Find administra las relaciones de conectividad entre los conjuntos de paradas; y el Grafo representa las conexiones de transporte y ejecuta los algoritmos necesarios para calcular recorridos.
Por lo tanto, no se trata de estructuras implementadas de manera aislada, sino de componentes complementarios que colaboran en el funcionamiento general del sistema. Su elección responde a las características de la red de transporte y a la necesidad de aplicar estructuras de datos propias, analizar su complejidad y aprovechar sus ventajas en las distintas operaciones de rutas.

Análisis de Complejidad de Estructuras

- **Tabla Hash**

Operación	Promedio	Peor caso	Alternativa nativa
Insertar	O(1)	O(n)	O(1)
Buscar	O(1)	O(n)	O(1)
Eliminar	O(1)	O(n)	O(1)
Recorrer	O(capacidad + n) = O(n)	O(n)	O(n)
Redimensionar	O(n)	O(n)	O(n)

La Tabla Hash ofrece un rendimiento promedio constante para las operaciones principales, debido al uso de una función hash y al control del factor de carga. En la implementación, el factor de carga se mantiene por debajo de 0.75, lo que ayuda a reducir las colisiones y conservar cadenas de elementos cortas.

Sin embargo, si muchas claves generan el mismo índice, las operaciones de búsqueda, inserción y eliminación pueden llegar a O(n), ya que sería necesario recorrer los elementos de una misma cubeta. El redimensionamiento requiere O(n), porque es necesario distribuir nuevamente los elementos en el arreglo de mayor capacidad. Aunque esta operación no ocurre en cada inserción, su costo se distribuye entre las inserciones realizadas.
El cálculo de la función hash depende de la longitud del código. Como los códigos de las paradas tienen una longitud pequeña y acotada, este costo se considera constante para los datos utilizados en el proyecto.

- **Grafo dirigido ponderado**

El grafo representa las paradas como vértices y las rutas como aristas ponderadas. Para analizar su rendimiento se consideran las operaciones de administración de conexiones y los algoritmos de recorrido y búsqueda de caminos.

Operación o algoritmo	**Complejidad**	**Explicación**
Buscar un vértice	O(V)	Se recorre la lista enlazada de vértices hasta encontrar la parada.
Insertar un vértice	O(V)	Se verifica que el vértice no exista antes de agregarlo.
Insertar una arista	O(V + E)	Se localizan los vértices y se agrega la conexión a la lista de adyacencia.
Recorrer vecinos	O(grado del vértice)	Se recorren las conexiones de un vértice específico.
BFS	O(V + E)	Visita los vértices y las aristas alcanzables utilizando una cola auxiliar.
DFS	O(V + E)	Explora los vértices y las conexiones mediante recorrido recursivo.
Dijkstra	O((V + E) log V)	Utiliza un montículo mínimo para seleccionar el siguiente vértice con menor distancia.

Se utiliza una lista de adyacencia para representar las conexiones del sistema, ya que una parada normalmente se conecta con una cantidad limitada de otras paradas. Esta representación requiere O(V + E) de espacio, lo que resulta más conveniente para una red dispersa que una matriz de adyacencia, cuya ocupación sería $O(V^2)$.

Los algoritmos BFS y DFS presentan una complejidad O(V + E), porque recorren los vértices y las aristas de la red. Por su parte, Dijkstra utiliza un montículo mínimo para encontrar el recorrido de menor tiempo o costo, alcanzando una complejidad de O((V + E) log V).

Una consideración de la implementación es que la búsqueda de un vértice se realiza mediante una lista enlazada, por lo que tiene un costo O(V). Este comportamiento debe tenerse en cuenta al analizar las operaciones que necesitan localizar vértices antes de trabajar con sus conexiones.

- **Union Find**

Esta estructura permite administrar la conectividad entre las paradas mediante conjuntos disjuntos. Su implementación utiliza compresión de camino y unión por rango para optimizar las operaciones.

Operación	Complejidad	Alternativa
Crear conjuntos	O(P)	Inicialización de arreglos
Buscar representante (Find)	O(α(P)) amortizado	BFS o DFS: O(V + E) por consulta
Unir conjuntos (Union)	O(α(P)) amortizado	No existe una estructura Union-Find equivalente directa en .NET
Consultar conectividad	O(α(P)) amortizado	BFS o DFS: O(V + E)
Reconstruir estructura	O(P + R·α(P))	Recorrer nuevamente las conexiones del grafo

Las operaciones principales de Union-Find presentan una complejidad amortizada prácticamente constante gracias a la compresión de camino y la unión por rango. Esto

permite consultar si dos paradas pertenecen al mismo conjunto sin recorrer nuevamente todas las conexiones del grafo.

No obstante, esta estructura no permite separar directamente dos elementos que ya fueron unidos. Por esta razón, cuando se eliminan rutas, es necesario reconstruir los conjuntos utilizando las conexiones que permanecen activas. Este procedimiento tiene una complejidad de $O(P + R \cdot \alpha(P))$.

- **Trie**

El Trie se utiliza para almacenar nombres y códigos de paradas, facilitando las búsquedas por prefijo y el autocompletado.

Operación	Complejidad	Alternativa
Insertar	$O(m \cdot \sigma)$	Lista: O(m) si se agrega directamente, sin comprobar duplicados
Buscar palabra completa	$O(m \cdot \sigma)$	Lista: $O(N \cdot m)$
Buscar por prefijo	$O(m \cdot \sigma)$	Lista: $O(N \cdot m)$
Eliminar	Depende de la implementación	Lista: $O(N \cdot m)$ para localizar y eliminar
Obtener sugerencias	Depende de la cantidad de resultados	Lista: $O(N \cdot m)$,

El Trie permite realizar búsquedas por prefijos sin comparar individualmente todos los nombres almacenados. Su rendimiento depende principalmente de la longitud del texto y de la organización de los nodos.

En la implementación del proyecto, cada nodo utiliza referencias al primer hijo y al siguiente hermano. Como los hermanos se mantienen ordenados alfabéticamente, la búsqueda de un carácter puede requerir recorrer varios hermanos hasta encontrarlo. Por esta razón, se considera el factor σ en la complejidad de las operaciones.

En comparación con una lista, que necesita revisar los nombres almacenados para encontrar coincidencias, el Trie ofrece una organización más adecuada para el autocompletado y las consultas por prefijo.

Estructuras y sus Ventajas

Estructura	Operación principal	Complejidad
Tabla Hash	Buscar una parada por código	O(1) promedio
Grafo	Recorrer la red mediante BFS o DFS	O(V + E)
Grafo	Encontrar la ruta de menor costo con Dijkstra	O((V + E) log V)
Union-Find	Consultar conectividad	O(α(P)) amortizado
Trie	Buscar nombres o códigos por prefijo	O(m·σ)

Las estructuras implementadas presentan diferentes características de rendimiento y cada una está orientada a resolver una operación específica del sistema. La Tabla Hash optimiza las consultas por código, el Grafo permite representar y recorrer la red de transporte, Union-Find facilita las consultas de conectividad y Trie mejora la búsqueda mediante prefijos. La combinación de estas estructuras permite desarrollar un sistema organizado y eficiente, aprovechando las ventajas de cada una y considerando sus respectivas limitaciones.